

Operating Systems

Rowan University

Overview

1. Computer Systems Analysis Review
2. Paging Algorithms

Test Your Understanding

What happens when the Xinu scheduler reschedules a process?

- Main things: update process table, context switching the CPU
- Old process's state is changed from `PR_CURR` to `PR_READY`
- Old process is inserted into the `readylist`
- The new process is dequeued (i.e. pulled) from the `readylist`
- The `currpri` is set to this new process's ID
- The state of the new process is set to `PR_CURR`
- The OS switched the context for the CPU from the old process to the new one
- `preempt` counter is reset to `QUANTUM`

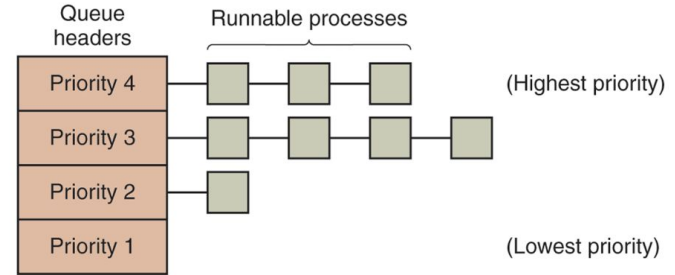
```
/* Decrement the preemption counter, and reschedule when the */  
/* remaining time reaches zero */  
  
if((--preempt) <= 0) {  
    preempt = QUANTUM;  
    resched();  
}
```

Test Your Understanding

What is readylist in the Xinu operating system source code? What information is contained inside of it?

- A list of runnable processes
- A priority queue of queues
- Stored as a linked list
- Each item in the list contains:
 - Reference to next, prev process
 - Key representing it's priority

Figure 2-44



A scheduling algorithm with four priority classes.

Test Your Understanding

How does the operating system protect the hardware from malicious behavior in applications, such as reading/writing areas of memory used by other running processes?

- Partitioning memory for processes, creating separate address spaces
 - A process trying to access a memory address outside its address space will fault the system
- User and Kernel space, privileges
- Protection bits (e.g. `rw-rw-rwx`)
- Protection Domains

Operating System Analysis

Course Objective:

- *Evaluate a computer-based system, process, component, or program*

Today's Activity:

- Work as a team
- Evaluate OS system for putting a process to sleep for a period of time
- Study the Xinu OS source code
- Present your analysis in the form of a written report

Review: Operating System Analysis

Report Specifications:

- PDF Document that answers the following questions:
 - How does the OS put a process to sleep?
 - What happens when a process is put to sleep by the OS?
 - How does the OS know how much time has passed?
 - How does the OS wake up a process?
 - What happens when a process is woken up by the OS?
- Document should have 5 headings, one for each question

Getting Started:

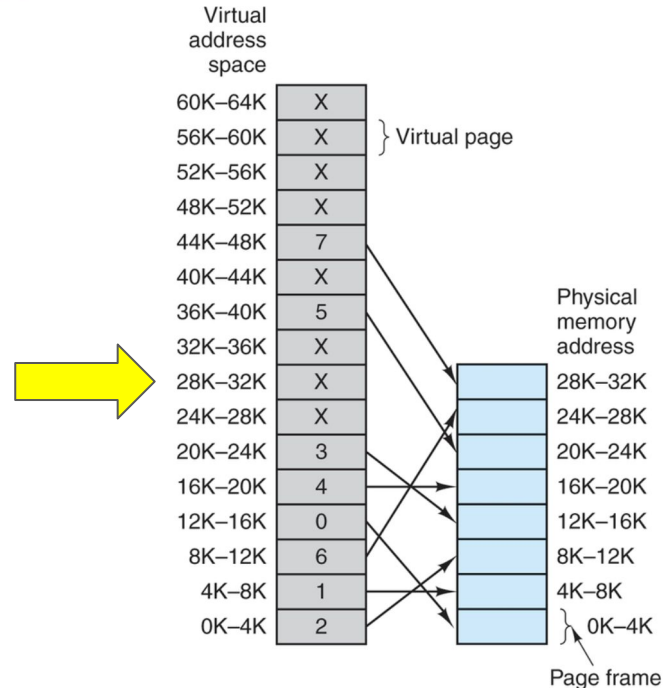
- Review clock.h to understand how time is tracked
- Observe the clkdisp.S, understand how interrupts trigger to update the clock time
- Review:
 - system/initialize.c
 - system/clkhandler.c
 - system/sleep.c
 - system/unsleep.c

Paging

- **Virtual Page** - Logical block of virtual memory
 - Page
- **Page Frame** - Units of Physical Memory
 - Frame

What happens if the program references an unmapped address?

Figure 3-9



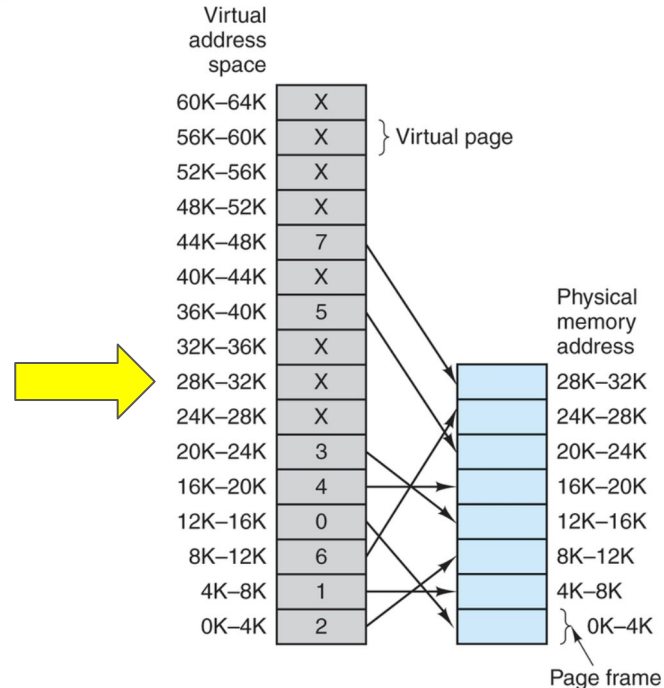
The relation between virtual addresses and physical memory addresses is given by the **page table**. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K-8K really means 4096-8191 and 4K-8K means 8192-12287.

Page Fault

Page Fault - The MMU sees an unmapped page and causes the CPU to *trap* to the operating system

1. The OS picks a page *frame* and writes it back to disk
2. Then, fetches the page that was just referenced into the page frame, updates the map
3. Restarts the *trapped* instruction

Figure 3-9



The relation between virtual addresses and physical memory addresses is given by the **page table**. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K-8K really means 4096-8191 and 4K-8K means 8192-12287.

Optimal Page Replacement

Problem: At the time of the page fault, the operating system has no way of knowing when each of the pages will be referenced next.

Figure 3-21

Algorithm	Comment
Optimal	Not implementable, but useful as a benchmark
NRU (Not Recently Used)	Very crude approximation of LRU
FIFO (First-In, First-Out)	Might throw out important pages
Second chance	Big improvement over FIFO
Clock	Realistic
LRU (Least Recently Used)	Excellent, but difficult to implement exactly
NFU (Not Frequently Used)	Fairly crude approximation to LRU
Aging	Efficient algorithm that approximates LRU well
Working set	Somewhat expensive to implement
WSClock	Good efficient algorithm

Page replacement algorithms discussed in the text.

Page Table

Figure 3-10

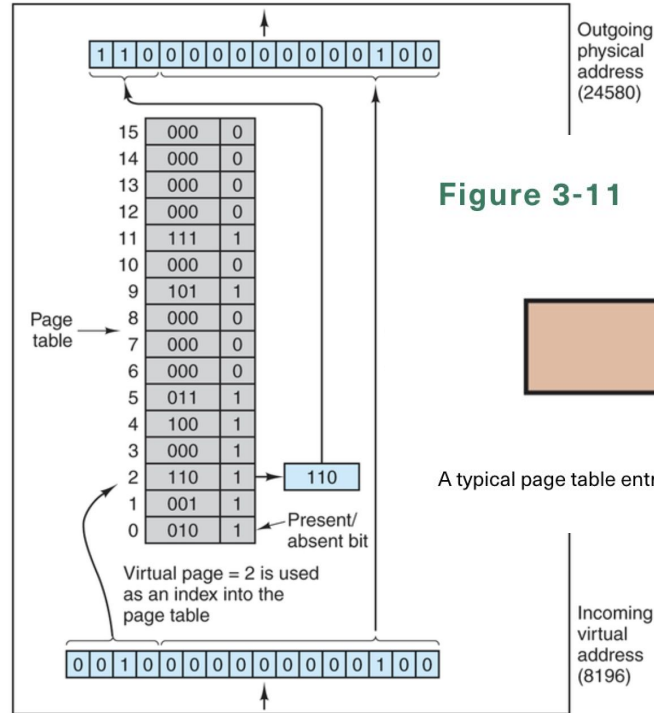
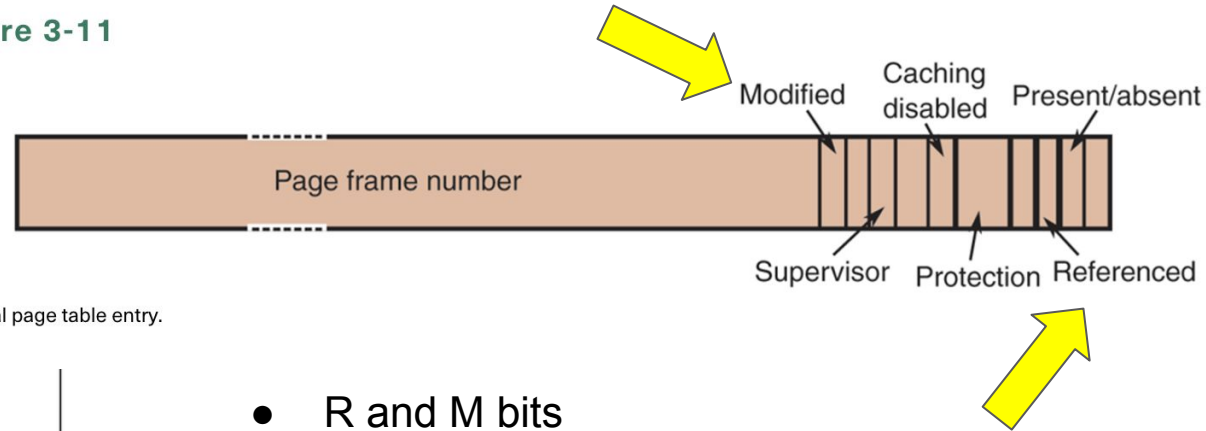


Figure 3-11

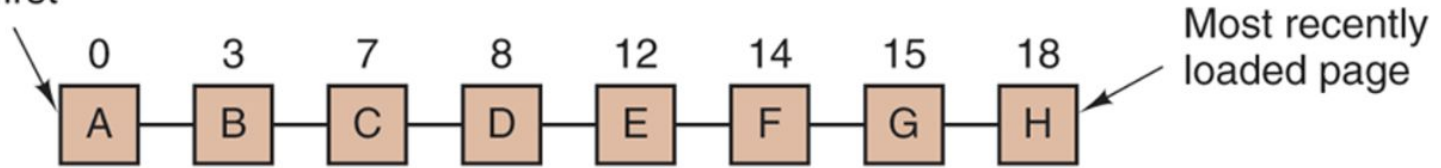


- R and M bits
- Has this been referenced?
- Has this been modified?

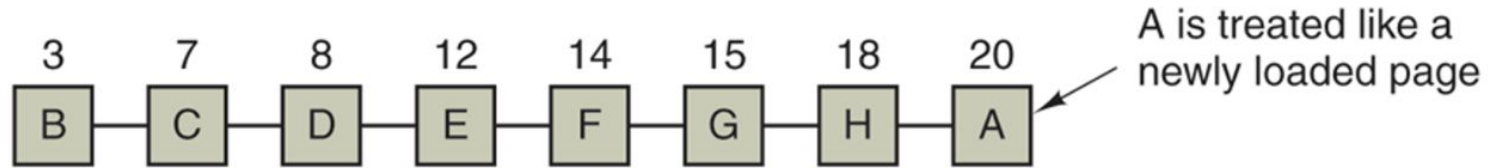
Second-Chance Page Replacement

Figure 3-15

Page loaded first



(a)

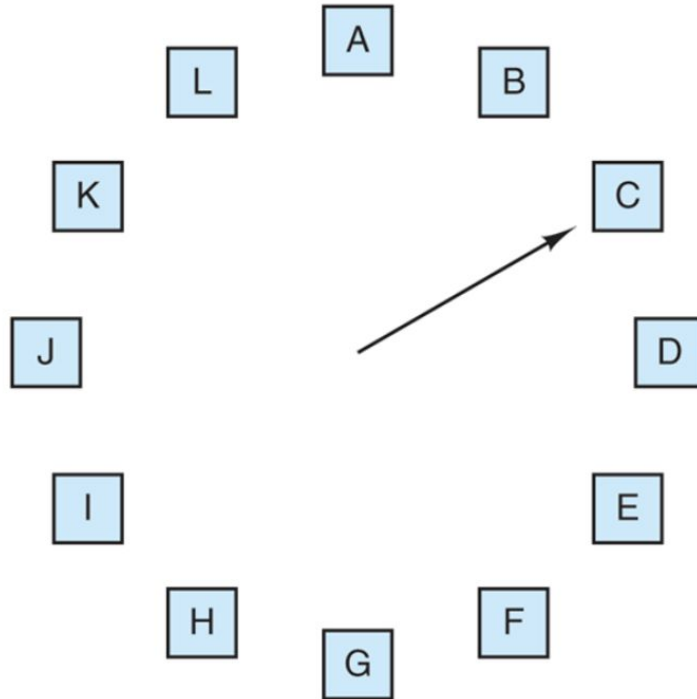


(b)

Operation of second chance. (a) Pages sorted in FIFO order. (b) Page list if a page fault occurs at time 20 and A has its *R* bit set. The numbers above the pages are their load times.

Clock Page Replacement Algorithm

Figure 3-16



When a page fault occurs, the page the hand is pointing to is inspected. The action taken depends on the R bit:

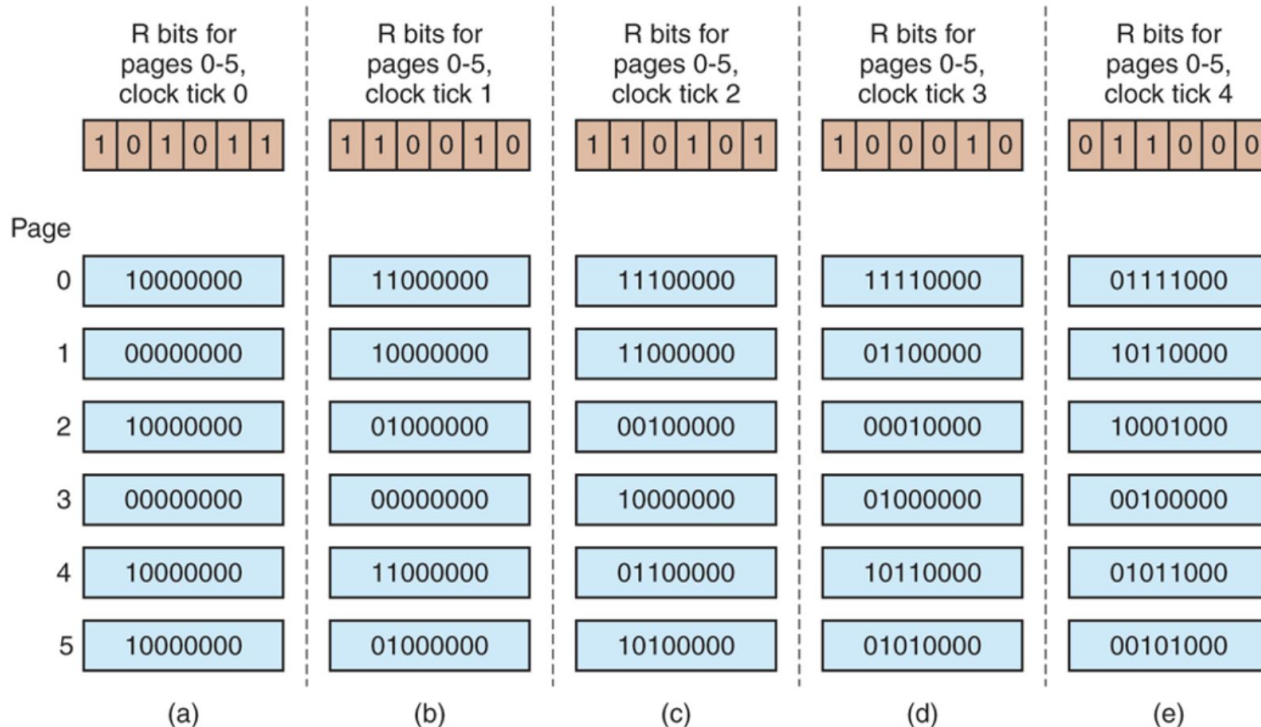
R = 0: Evict the page

R = 1: Clear R and advance hand

The clock page replacement algorithm.

Least Recently Used (LRU) Algorithm

Figure 3-17



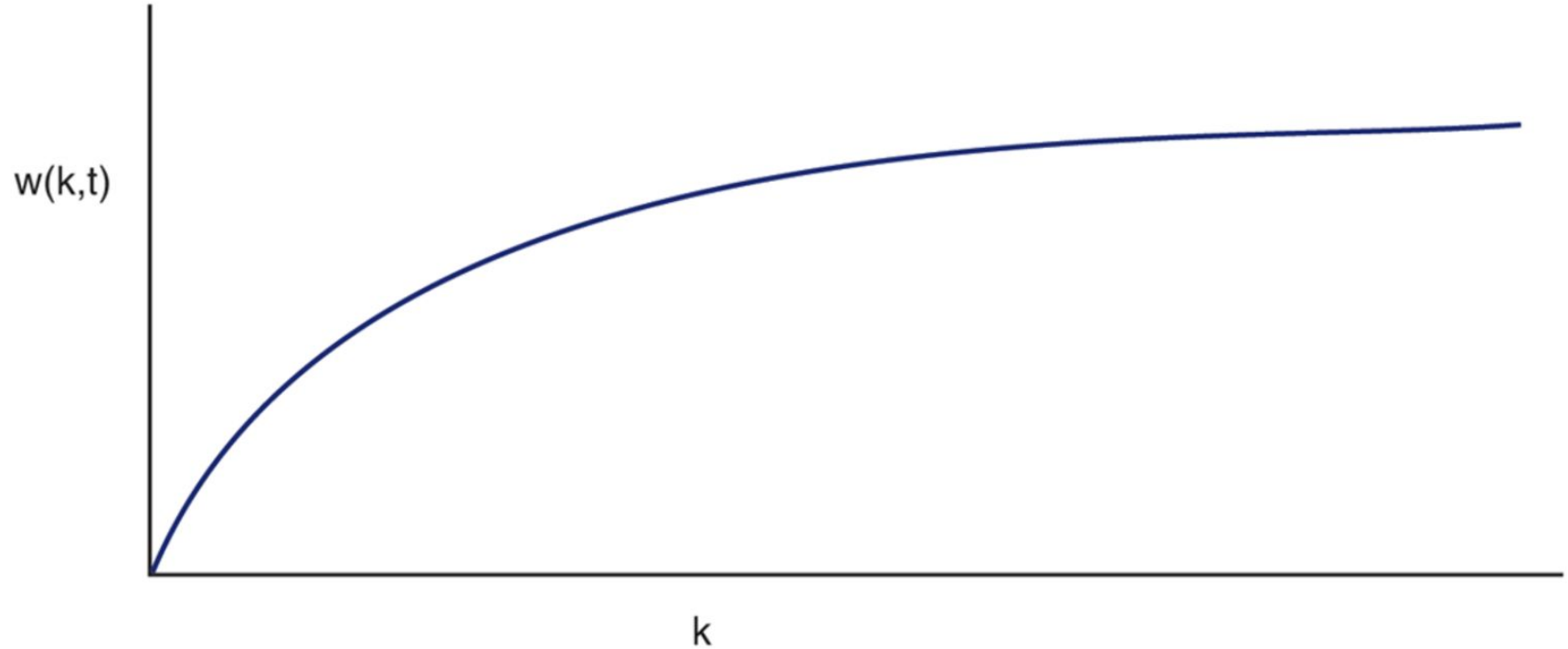
The aging algorithm simulates LRU in software. Shown are six pages for five clock ticks. The five clock ticks are represented by (a) to (e).

Working Set Model

- **Demand Paging** - Pages are loaded only on demand
- **Locality of Reference** - Process references a relatively small fraction of pages
 - Page references are cluster, memory access is not uniformly random
- **Working Set** - The pages a process is currently using
- **Observation** - If the process's working set is already loaded then the process will not generate an page faults
- **Prepaging** - Loading pages before letting a process run

Working Step Model

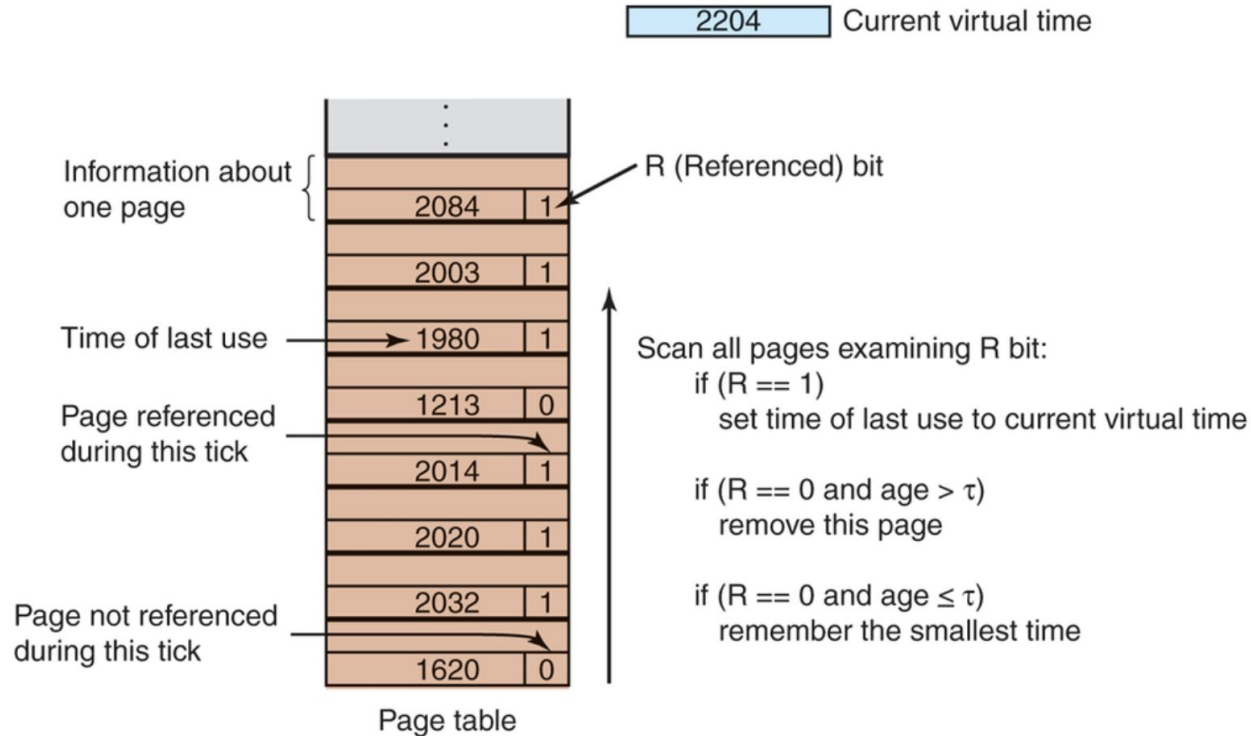
Figure 3-18



The working set is the set of pages used by the k most recent memory references. The function $w(k, t)$ is the size of the working set at time t .

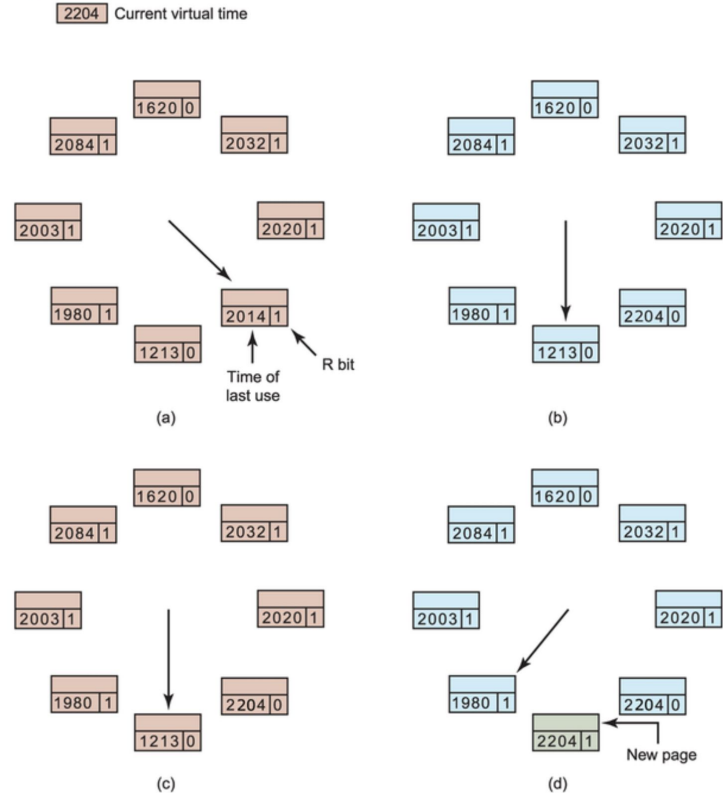
Working Set Algorithm

Figure 3-19



WSClock Page Replacement

Figure 3-20



Operation of the WSClock algorithm. (a) and (b) give an example of what happens when $R=1$. (c) and (d) give an example of $R=0$.